Laboratory Record

CSA4029 Reinforcement and Representation Learning

B21+E14+E22, BL2025260500511,

Winter Semester:2025-2026

Name of the student: Vinay Kumar

Register Number: 23BAI10125

Exp No: 1	Frozen Lake (Q-Learning)
31.03.2026	

Aim:

Q-Learning implementation on the classic Frozen Lake grid problem — an agent learns to navigate from start to goal while avoiding holes.

Procedure:

Step 1: Initialization

Before the agent takes a single step, it creates a Q-Table. This is essentially a cheat sheet or a spreadsheet.

- Rows: Every possible square (state) on the lake (0 to 15).
- Columns: Every possible move (Up, Down, Left, Right).
- Starting Values: Every cell in the table starts at 0 because the agent knows nothing about the world yet.

Step 2: The Action (Epsilon-Greedy)

In every turn, the agent must decide: *"Do I follow my table, or do I try something new?"*

- Exploration: If a random number is less than Epsilon, the agent picks a random direction. This is crucial early on so the agent can find the goal.
- Exploitation: If the number is higher, the agent looks at its Q-Table for the current square and picks the direction with the highest value.

Step 3: Observation

The agent performs the action (e.g., moves Right). The environment then tells the agent two things:

1. The Reward (R): Did you hit the goal (+1), fall in a hole (0), or just move to another ice tile (0)?
2. The New State (s'): Which square are you standing on now?

Step 4: The "Learning" (The Bellman Equation)

This is the heart of the algorithm. The agent updates the Q-Table for the square it *just left* using this logic:

> New Value = Old Value + Learning Rate × [Reward + (Discount × Best Future Value) - Old Value]

- Reward: Immediate gratification.
- Best Future Value: The agent looks at the *new* square and sees which move there has the highest potential. This allows the "glory" of the goal to trickle backward to the squares leading up to it.
- Discount Factor (gamma): A value (like 0.95) that makes the agent care slightly less about rewards that are far away than rewards that are immediate.

Step 5: Epsilon Decay

As the "Episodes" (games) progress, we slowly lower the Epsilon value.

- **Early game:** The agent is a wanderer, falling into holes and occasionally bumping into the goal by luck.
- **Late game:** The agent becomes a master. Because Epsilon is low, it stops guessing and starts following the high values it has recorded in its Q-Table.

Step 6: Convergence

After thousands of rounds, the values in the Q-Table stabilize. If you look at the table, you will see a "path" of high numbers leading from the start, around the holes, and directly to the goal.

Summary of the "Mental Model"

If the agent is at Square 14 (right next to the Goal at 15):

1. It moves Right and gets a reward of **1**.
2. It updates the Q-Table for Square 14/Action: Right to be high.
3. Next time it's at Square 13, it looks at Square 14, sees that high value, and realizes, *"Hey, if I go to 14, I'm close to a win!"*
4. The knowledge "flows" backward until the Start square knows exactly which way to turn.

Output:

```python
import numpy as np
import random

# — Environment: 4×4 FrozenLake ————————————————————————————
# S = Start, F = Frozen (safe), H = Hole (game over), G = Goal
GRID = [
    ['S', 'F', 'F', 'F'],
    ['F', 'H', 'F', 'H'],
    ['F', 'F', 'F', 'H'],
    ['H', 'F', 'F', 'G'],
]
ROWS, COLS = 4, 4
NUM_STATES  = ROWS * COLS          # 16 states
NUM_ACTIONS = 4                    # 0=Up 1=Down 2=Left 3=Right

ACTIONS = {0: (-1, 0), 1: (1, 0), 2: (0, -1), 3: (0, 1)}
ACTION_NAMES = {0: '↑', 1: '↓', 2: '←', 3: '→'}

def state_to_pos(s):
    return s // COLS, s % COLS

def pos_to_state(r, c):
    return r * COLS + c

def step(state, action):
    """Return (next_state, reward, done)."""
    r, c = state_to_pos(state)
    dr, dc = ACTIONS[action]
    nr, nc = max(0, min(ROWS-1, r+dr)), max(0, min(COLS-1, c+dc))
    ns = pos_to_state(nr, nc)
    cell = GRID[nr][nc]
    if cell == 'H':
        return ns, -1.0, True   # fell in hole
    if cell == 'G':
        return ns, +1.0, True   # reached goal
    return ns, 0.0, False       # safe step

# — Q-Learning ————————————————————————————————————————————
# Hyper-parameters
ALPHA   = 0.1    # learning rate
GAMMA   = 0.99   # discount factor
EPSILON = 1.0    # exploration rate (decays over time)
EPS_MIN = 0.01
EPS_DECAY = 0.995
EPISODES = 5_000

Q = np.zeros((NUM_STATES, NUM_ACTIONS))  # Q-table: 16×4

def choose_action(state, eps):
    """ε-greedy policy."""
    if random.random() < eps:
```

```python
def train():
    global EPSILON
    rewards_per_episode = []

    for ep in range(EPISODES):
        state = 0  # always start at S (top-left)
        total_reward = 0

        while True:
            action = choose_action(state, EPSILON)
            next_state, reward, done = step(state, action)

            # — Bellman update ————————————————————————
            td_target = reward + GAMMA * np.max(Q[next_state]) * (not done)
            td_error  = td_target - Q[state, action]
            Q[state, action] += ALPHA * td_error
            # ————————————————————————————————————————

            state = next_state
            total_reward += reward

            if done:
                break

        # Decay exploration
        EPSILON = max(EPS_MIN, EPSILON * EPS_DECAY)
        rewards_per_episode.append(total_reward)

    return rewards_per_episode

# — Run training ——————————————————————————————————————————
print("Training Q-Learning agent on FrozenLake...\n")
rewards = train()

# — Print results ————————————————————————————————————————
window = 500
avg = np.convolve(rewards, np.ones(window)/window, mode='valid')
print(f"Average reward over last {window} episodes: {avg[-1]:.3f}")
print(f"Final ε (exploration rate): {EPSILON:.4f}\n")

# — Show learned policy ——————————————————————————————————
print("Learned Policy (best action per cell):")
print("┌────┬────┬────┬────┐")
for r in range(ROWS):
    row_str = "|"
    for c in range(COLS):
        cell = GRID[r][c]
        if cell in ['H', 'G', 'S']:
            row_str += f" {cell:2} |"
        else:
            s = pos_to_state(r, c)
            best = int(np.argmax(Q[s]))
```

Untitled-2.ipynb

Generate + Code + Markdown | Run All Restart Clear All Outputs | Jupyter Variables Outline ···

Python 3.12.4

```python
    print("└──────┴──────┴──────┴──────┘")

    # — Show Q-table ————————————————————————————————————————
    print("\nQ-Table (rows=states 0–15, cols=↑↓←→):")
    print(f"{'State':>6}  {'↑':>7}  {'↓':>7}  {'←':>7}  {'→':>7}")
    print("-" * 42)
    for s in range(NUM_STATES):
        r, c = state_to_pos(s)
        label = GRID[r][c]
        vals  = "  ".join(f"{v:7.3f}" for v in Q[s])
        print(f"  [{label}]{s:2}  {vals}")

    # — Greedy rollout ——————————————————————————————————————
    print("\nGreedy rollout (ε=0):")
    state, path = 0, [0]
    for _ in range(50):
        action = int(np.argmax(Q[state]))
        state, _, done = step(state, action)
        path.append(state)
        if done:
            break

    r, c = state_to_pos(path[-1])
    outcome = GRID[r][c]
    print(f"Path: {' → '.join(str(s) for s in path)}")
    print(f"Outcome: {'Goal reached!' if outcome == 'G' else 'Fell in a hole.'}")
```

[2] ✓ 28.0s Python

```
Training Q-Learning agent on FrozenLake...

Average reward over last 500 episodes: -1.000
Final ε (exploration rate): 0.0100

Learned Policy (best action per cell):

┌───┬───┬───┬───┐
│ S │ ↑ │ ↑ │ ↑ │
├───┼───┼───┼───┤
│ ↑ │ H │ ↑ │ H │
├───┼───┼───┼───┤
│ ↑ │ ↓ │ ↑ │ H │
├───┼───┼───┼───┤
│ H │ ↑ │ ↑ │ G │
└───┴───┴───┴───┘

Q-Table (rows=states 0–15, cols=↑↓←→):
 State        ↑        ↓        ←        →
==========================================
  [S] 0    0.000    0.000    0.000    0.000
  [F] 1    0.000   -1.000    0.000    0.000
  [F] 2    0.000    0.000    0.000    0.000
  [F] 3    0.000   -1.000    0.000    0.000
  [F] 4    0.000    0.000    0.000   -1.000
  [H] 5    0.000    0.000    0.000    0.000
...
```

0 files and 12 cells to analyze | Start JSON Server | Spaces: 4 LF {} Cell 1 of 1